PBP • PEMROGRAMAN BERORIENTASI PLATFORM

Panduan Latihan Demo CPMK 2

Apple Directory — Sistem Informasi Kepegawaian

Buku latihan untuk menyiapkan demo aplikasi, membuktikan alur di Postman, dan menjawab pertanyaan dosen dengan runtut.

Target demo	Yang perlu dibuktikan
1. Aplikasi React	Login, lihat direktori, admin mengelola data, user hanya membaca.
2. REST API	Endpoint HR mendukung baca dan operasi CRUD.
3. JWT	Token dibutuhkan untuk endpoint terlindungi; token dapat refresh dan logout.
4. Otorisasi role	Admin boleh menulis; user mendapat 403 untuk operasi tulis.

Cara berlatih
Jalankan aplikasinya sambil mengikuti urutan halaman 2–7. Isi data demo sementara saja, dan hapus kembali setelah pembuktian.

Kredensial demo: admin@apple.com / password123 • user@apple.com / password123

01 Persiapan sebelum presentasi

Lakukan persiapan ini sebelum dosen datang supaya waktu presentasi dipakai untuk menunjukkan fitur, bukan memperbaiki setup.

A. Jalankan aplikasi

Terminal — folder proyek

```
cd /home/xavierwjaya/Documents/PBP_MASYERE/apple-hr-system
npm run build
php artisan migrate --seed
php artisan serve --host=127.0.0.1 --port=8001
```

Biarkan terminal server tetap terbuka. Buka http://127.0.0.1:8001/login di browser. Database lokal proyek ini memakai MariaDB; pastikan service aktif dan koneksi .env benar. Jalankan php artisan db:show untuk memeriksa koneksi. Jalankan npm run build jika aset belum dibangun atau baru berubah.

Catatan database
migrate --seed menambahkan tabel yang belum ada dan mengisi akun serta contoh karyawan. Jangan gunakan migrate:fresh untuk latihan kecuali memang rela menghapus seluruh data database lokal.

B. Pastikan akun dan tampilan siap

- Admin: admin@apple.com — password123
- User: user@apple.com — password123
- Contoh data direktori: AAPL-101 dan AAPL-102. Jangan hapus data contoh; buat satu record demo sementara untuk latihan CRUD.
- Siapkan browser di halaman login, terminal server, dan Postman. Pastikan tidak ada proses lain memakai port 8001.

C. Siapkan Postman satu kali

1. Import collection — di Postman klik Import → Files, pilih file postman/Apple-HR-CPMK2.postman_collection.json dari folder proyek.

2. Periksa variables — di collection, base_url awalnya http://127.0.0.1:8001/api/v1. Skrip menyimpan token dan employeeDbId otomatis.

3. Ikuti folder 1 sampai 6 berurutan — Login Admin → Admin CRUD → Login User → Bukti User Read Only → Login Admin Lagi → Hapus Record Demo.

4. Lihat status dan tab Test — request berjalan dengan Send; response menampilkan status code dan JSON. Skrip collection juga memberi tanda test lulus.

Penting
Token berubah saat login user/admin. Folder terakhir hanya menghapus record sementara yang dibuat folder Admin CRUD. Jangan jalankan delete sebelum tes user selesai.

02 Demo aplikasi React

Urutan ini menunjukkan pengalaman pengguna dan perbedaan hak akses sebelum masuk ke pembuktian API.

Urutan	Aksi di layar	Bukti yang disebutkan
1	Buka /login. Tunjukkan halaman masuk Apple Directory.	Ini frontend React bertema HR.
2	Masuk sebagai admin@apple.com / password123.	Header menampilkan identitas dan label Administrator.
3	Tunjukkan direktori, jumlah karyawan, tim, pencarian.	Admin dapat membaca daftar dan mencari data.
4	Klik Add employee. Isi record sementara, misalnya AAPL-DEMO-01.	Admin memiliki fitur Create.
5	Edit judul / divisi record itu, simpan.	Admin memiliki fitur Update.
6	Hapus hanya record AAPL-DEMO-01.	Admin memiliki fitur Delete; data contoh tetap aman.
7	Sign out. Masuk sebagai user@apple.com / password123.	Label akses berubah menjadi Team member / Read only.
8	Tunjukkan direktori. Perhatikan tombol tambah/edit/hapus tidak ada.	User masih bisa membaca data, tetapi UI menyembunyikan aksi tulis.

Skrip bicara yang bisa dihafalkan

Pembuka
“Aplikasi ini adalah sistem informasi kepegawaian bertema Apple. Frontend dibuat dengan React dan data karyawan dikelola melalui REST API Laravel.”

Perbedaan role
“Admin memiliki akses CRUD. User hanya membaca. Tombol disembunyikan di frontend, lalu saya buktikan sebentar lagi bahwa API juga menolak request tulis dari user.”

Catatan istilah
Untuk demo ini, role user berarti staf pembaca direktori. Role admin berarti pengelola HR.

Tips: jangan menghabiskan waktu mengetik panjang. Siapkan data sementara dan urutan klik sebelumnya; tetap tunjukkan satu kali create, update, dan delete secara nyata.

03 Postman: login dan akses baca

Buat request-request berikut di collection. Gunakan {{base_url}} agar URL mudah dipakai ulang. Di tab Authorization pilih “Inherit auth from parent” untuk endpoint yang memerlukan JWT.

Request 1 — Login sebagai admin

POST {{base_url}}/auth/login • Body > raw > JSON

```
{
  "email": "admin@apple.com",
  "password": "password123"
}
```

Hasil yang diharapkan: 200 OK; respons berisi token, token_type Bearer, expires_in, dan user.role = admin. Di tab Scripts > Post-response tempel skrip ini agar token otomatis tersimpan:

Post-response script — simpan token login

```
const data = pm.response.json();
pm.collectionVariables.set('token', data.token);
pm.collectionVariables.set('role', data.user.role);
pm.test('Login admin berhasil', () => pm.response.to.have.status(200));
pm.test('Role admin benar', () => pm.expect(data.user.role).to.eql('admin'));
```

Request 2 — Profil dari token

GET {{base_url}}/auth/me • Authorization: Bearer {{token}}

```
Tidak ada body
```

Hasil: 200 OK dan data user admin. Ini membuktikan token dapat mengautentikasi pengguna.

Request 3 — Baca daftar karyawan

GET {{base_url}}/employees • Authorization: Bearer {{token}}

```
Tidak ada body
```

Hasil: 200 OK dan daftar dalam bentuk JSON di properti data. Data karyawan mencakup id, employee_id, nama, role_title, team_division, location, dan salary.

Request 4 — Baca satu karyawan

GET {{base_url}}/employees/1 • Authorization: Bearer {{token}}

```
Tidak ada body
```

Ganti angka 1 dengan id yang ada pada respons daftar. Hasil: 200 OK dan detail satu karyawan.

Bukti yang diperlihatkan ke dosen
Klik request di Postman, tunjukkan method + URL, Bearer token, lalu status code dan response JSON. Jangan tampilkan atau kirim token ke orang lain; token berlaku seperti kredensial sementara.

04 Postman: buktikan CRUD sebagai admin

Pastikan token collection saat ini berasal dari login admin. Buat satu karyawan sementara dengan employee_id unik supaya aman dari duplikasi.

Request 5 — Create

POST {{base_url}}/employees • Authorization Bearer {{token}} • Body JSON

```
{
  "employee_id": "AAPL-POSTMAN-01",
  "nama": "Demo CPMK Dua",
  "role_title": "HR Demo Specialist",
  "team_division": "People Operations",
  "location": "Apple Park, Cupertino",
  "salary": 80000
}
```

Hasil: 201 Created. Respons data karyawan berisi id database. Tambahkan skrip untuk menyimpan id:

Post-response script — simpan id record baru

```
const body = pm.response.json();
pm.collectionVariables.set('employeeDbId', body.data.id);
pm.test('Record dibuat', () => pm.response.to.have.status(201));
```

Request 6 — Read record yang dibuat

GET {{base_url}}/employees/{{employeeDbId}}

```
Tidak ada body; Authorization Bearer {{token}}
```

Hasil: 200 OK; pastikan employee_id bernilai AAPL-POSTMAN-01.

Request 7 — Update

PUT {{base_url}}/employees/{{employeeDbId}} • Body JSON

```
{
  "employee_id": "AAPL-POSTMAN-01",
  "nama": "Demo CPMK Dua Updated",
  "role_title": "People Operations Analyst",
  "team_division": "People Operations",
  "location": "Apple Park, Cupertino",
  "salary": 85000
}
```

Hasil: 200 OK. Endpoint update memerlukan seluruh field wajib. Periksa nama/judul/salary di respons.

Request 8 — Delete (lakukan paling akhir)

DELETE {{base_url}}/employees/{{employeeDbId}}

```
Tidak ada body; Authorization Bearer {{token}}
```

Hasil: 204 No Content. Setelah itu GET ke id tersebut seharusnya memberi 404 Not Found. Record sementara sudah bersih.

Urutan yang aman
Create → simpan id → Read → Update → buktikan 403 untuk user (halaman 6) → login kembali sebagai admin → Delete record sementara.

05 Postman: bukti JWT dan pembatasan role

Lakukan tes user menggunakan record sementara yang baru dibuat. Login user akan mengganti token collection.

Login sebagai user

POST {{base_url}}/auth/login

```
{
  "email": "user@apple.com",
  "password": "password123"
}
```

Gunakan post-response script yang sama seperti login admin. Pastikan role yang tersimpan adalah user.

Bukti: user boleh membaca

GET {{base_url}}/employees • Authorization Bearer {{token}}

```
Hasil yang diharapkan: 200 OK
```

Bukti: user tidak boleh menulis

POST {{base_url}}/employees • Authorization Bearer {{token}} • Body sama seperti request Create

```
Hasil yang diharapkan: 403 Forbidden
```

Untuk pembuktian yang lebih kuat, coba juga PUT /employees/{{employeeDbId}} dengan seluruh field wajib, lalu DELETE /employees/{{employeeDbId}}. Keduanya harus memberi 403 Forbidden dan record sementara harus tetap ada.

Bukti: autentikasi memang diperlukan

GET {{base_url}}/employees • Authorization > No Auth (hapus Bearer token)

```
Hasil yang diharapkan: 401 Unauthorized
```

Bukti: kredensial salah

POST {{base_url}}/auth/login • password salah

```
Hasil yang diharapkan: 401 Unauthorized
```

Refresh dan logout token (opsional bila waktu cukup)

- Refresh: POST {{base_url}}/auth/refresh dengan Bearer token aktif. Harus 200; jalankan script penyimpan token agar token baru menggantikan token lama.
- Logout: POST {{base_url}}/auth/logout dengan token aktif. Harus 200. Coba lagi GET /auth/me dengan token tersebut; harus 401.

Jangan tertukar
401 = belum terautentikasi / token tidak valid. 403 = sudah terautentikasi, tetapi role tidak berhak melakukan aksi.

06 Skrip presentasi dan tanya-jawab

Contoh alur bicara sekitar 3–5 menit

1. Tujuan — “Aplikasi ini memenuhi topik CPMK 1 saya, yaitu sistem informasi kepegawaian, lalu dikembangkan untuk CPMK 2 dengan frontend React dan layanan API.”

2. Tunjukkan UI — “Saya login sebagai admin dan memperlihatkan operasi CRUD. Setelah logout saya login sebagai user; user tetap dapat melihat direktori, tetapi tidak memiliki tombol perubahan.”

3. Buktikan backend — “Sekarang saya tunjukkan endpoint API dan status responsnya di Postman. Admin mendapat 201 saat membuat data; user mendapat 403 untuk request tulis.”

4. Tutup — “Autentikasi menggunakan JWT. API memeriksa token dan role di backend, sehingga pembatasan tetap berlaku walaupun request dikirim langsung dari Postman.”

Pertanyaan dosen yang mungkin muncul

Pertanyaan	Jawaban ringkas
Apa bedanya autentikasi dan otorisasi?	Autentikasi memeriksa siapa pengguna lewat JWT. Otorisasi memeriksa izin berdasarkan role admin/user.
JWT dipakai di mana?	Client mengirim token di header Authorization: Bearer untuk endpoint yang memerlukan login.
Mengapa tombol user disembunyikan?	Agar UI sesuai hak akses. Keamanan tetap ditegakkan di server melalui middleware admin; tes Postman menunjukkan user mendapat 403.
Apa bukti user read-only?	GET daftar karyawan memberi 200, sedangkan POST/PUT/DELETE user memberi 403.
Apa arti status 401 dan 403?	401: token tidak ada/tidak valid. 403: pengguna dikenali tetapi tidak punya izin.
Bagaimana aplikasi menyimpan data?	Laravel menyediakan API. Data aplikasi ini tersimpan di MariaDB; React memanggil API untuk membaca dan mengubah record.

Checklist malam sebelum demo

- Server dapat dijalankan; halaman /login terbuka.
- Seeder menyediakan dua akun; hafal email dan password.
- Postman punya collection variables base_url, token, employeeDbId.
- Latih membuat dan menghapus satu record sementara; jangan sentuh data contoh.
- Sebelum tes role, login ulang sesuai role. Token admin dan user berbeda.
- Tutup dengan status code dan bukti request, bukan hanya menyebut fitur.

Kalau waktu presentasi pendek
Prioritaskan: login admin → satu siklus CRUD singkat → login user → GET 200 dan POST 403 di Postman.
Refresh/logout bisa dijelaskan jika dosen bertanya.

Endpoint ringkas: POST /auth/login • GET /auth/me • POST /auth/refresh • POST /auth/logout • GET /employees • GET /employees/{id} • POST /employees • PUT /employees/{id} • DELETE /employees/{id}